Virtual Threads

Season: 5 · Concurrency · Category: Concurrency · Level: Intermediate

Intent

- Lightweight threads for massive concurrency (Java 21+).

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Millions of cheap coroutines scheduled on carriers.

When to Use

- Concurrent / parallel workloads.
- The design force named in Intent shows up more than once.

When Not to Use

- Pure CPU-bound work already parallelized.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.concurrency.virtualthreads;

import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

/** Java 21+ virtual threads — prefer over hand-rolled pools for blocking work. */
public final class VirtualThreadDemo {
    private VirtualThreadDemo() {}

    public static int fanOut(int tasks) throws InterruptedException {
        var hits = new AtomicInteger();
        try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < tasks; i++) {
                pool.submit(hits::incrementAndGet);
            }
        }
        return hits.get();
    }
}
```

Modern Java 25 / Functional

- Executors.newVirtualThreadPerTaskExecutor() is the modern default for blocking tasks.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: concurrency/virtualthreads/
```

Pitfalls

- Prefer java.util.concurrent and virtual threads over custom pools.
- Always interrupt/shutdown cleanly.

Interview Q&A

- Q: Virtual threads vs platform threads? A: Virtual threads excel at blocking I/O concurrency; still bound by CPU for pure compute.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Lightweight threads for massive concurrency (Java 21+).
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Millions of cheap coroutines scheduled on carriers. [0:40] Intent — State the design force in one se...